3DGenerateFlow Project Introduction

1. Background

With the maturation of consumer 3D printing, full-color printing (Mimaki / Stratasys / PolyJet), and AI generation technologies, ordinary users and small studios increasingly want to turn a photo of an object, pet, or person into a printable full-color 3D or 2.5D memento. Existing tools suffer from several barriers:

- Multi-view capture and modeling workflows are complex.
- Stylization, 3D generation, and print checking require switching between multiple tools.
- Core model inference relies on closed-source APIs, with high cost and low controllability.
- There is a lack of end-to-end creative tools for "photo to print."

3DGenerateFlow aims to solve these problems: users upload a photo and describe the desired style in one sentence; the system automatically completes multi-view synthesis, 3D generation / 2.5D relief, texture mapping, and print checking on a local AMD Radeon GPU, and finally outputs a model file ready for 3D printing.

---

2. Target Users and Application Scenarios

Target Users

- **Individual creators**: Want to make 3D-printed mementos of themselves, pets, or figure prototypes.
- **Small design studios / e-commerce sellers**: Batch produce personalized IP derivatives, fridge magnets, commemorative coins, and relief medals.
- **3D printing service providers**: Quickly generate printable files from customer photos to shorten delivery cycles.
- **Educators / makerspaces**: Use for introductory teaching and demos of AI + 3D printing.

Application Scenarios

- Pet / person full-body 3D figures in realistic, cartoon, low-poly, voxel, and other styles.
- Photo reliefs, lithophanes, commemorative coins, and silhouette pendants.
- Commercial visual design: quickly turn product photos into 3D display models or 3D-printed prototypes.
- Social media content creation: generate shareable cards with 3D previews.

Social Value

- Lowers the barrier to 3D content creation, enabling non-professional users to participate in personalized manufacturing.
- Supports local AMD GPU inference, reducing reliance on foreign closed-source APIs and improving data privacy and controllability.
- Provides a reference implementation for engineering open-source AI 3D generation pipelines (Hunyuan3D-2, Zero123) on the ROCm ecosystem.

---

3. System Architecture

```
                    Web UI (React + R3F)
Upload · AI Style Planning · Lazy Canvas Flow · 3D Preview · DL

                    HTTP/WebSocket
```

```
                FastAPI + Celery (Eager Mode)
/upload · /agent/plan · /agent/execute · /jobs · /health/gpu

ROCm Adapter          Hunyuan3D-2           3D Director
· SD img2img          · 2D → 3D             · Style Plan
· Zero123             · 2mv multi-view      · Task Sched
· Depth V2            · Texture fallback    · Memory/Chat

       Print Check / Export
· Scale / center / base plane
· Volume / bbox / watertight
· GLB / STL output
```

Tech Stack

- **Frontend**: React 19 + TypeScript + Tailwind CSS + Vite + React Three Fiber / Drei
- **Backend**: Python 3.12 + FastAPI + Celery (Eager synchronous mode for demo) + SQLAlchemy + SQLite
- **AI Inference**: PyTorch 2.5.1 + ROCm 6.1 + Diffusers + Transformers + Trimesh
- **Core Models**:
  - Stable Diffusion v1.5 (img2img stylization)
  - Zero123-xl (multi-view synthesis)
  - Depth Anything V2 Small (depth estimation)
  - Hunyuan3D-2 / Hunyuan3D-2mv (image-to-3D)
- **Runtime**: AMD Radeon GPU (gfx1100, 48 GB VRAM) + ROCm open software stack

---

4. Models and Algorithms

4.1 Image-to-Image Stylization (Stable Diffusion img2img)

- Run `runwayml/stable-diffusion-v1-5` locally on ROCm.
- Resize the original image so the longest side is 1024 px while preserving aspect ratio.
- Concatenate prompts from the style catalog (realistic, cartoon, low-poly, voxel, clay, sketch, etc.).
- Output `styled_preview.png`, also used as the front reference for 3D generation.

4.2 Multi-View Synthesis (Zero123)

- Use `ashawkey/zero123-xl-diffusers` to generate front / right / back / left views from a single front-facing image.
- 512×512 output; input is center-cropped to a square for view consistency.
- Provides input for subsequent Hunyuan3D-2mv multi-view 3D generation.

4.3 Depth Estimation (Depth Anything V2)

- Run `depth-anything/Depth-Anything-V2-Small-hf` on ROCm.

- Output a grayscale depth map as the height field for 2.5D relief.

4.4 Image-to-3D (Hunyuan3D-2 / Hunyuan3D-2mv)

- **Hunyuan3D-2**: single-view 3D mesh generation, suitable for quick validation.
- **Hunyuan3D-2mv**: multi-view (front / right / back / left) input, better geometric consistency; this is the main pipeline.
- Inference with `torch.float16` on ROCm; the 48 GB VRAM can load the full model.
- Raw output is normalized coordinates; the post-processing module scales uniformly by style `target_height_mm` (e.g. 80 mm), centers, and places the base on Z=0.

4.5 Full-Color Texture

- Prefer Hunyuan3D-2's own texture module; if CUDA extensions fail to compile on ROCm, automatic fallback:
  - 3D models: front-projection UV, pasting the stylized reference image onto the mesh front.
  - 2.5D relief: 1:1 pixel UV, using the original image as texture to generate a colorful GLB; STL is also exported for monochrome printing.

4.6 Print Checking

- Use Trimesh to compute mesh volume, bounding box dimensions, and watertight status.
- After scaling, ensure the base is flat and dimensions are in mm; output `print_report` for frontend display.

---

5. AMD Radeon GPU / ROCm Adaptation

5.1 Hardware Environment

- GPU: AMD Radeon Graphics (Navi31 / gfx1100)
- Compute Units: 96 CU
- VRAM: 48 GB GDDR6
- Driver: amdgpu 6.16.13
- ROCm: PyTorch 2.5.1+rocm6.1

5.2 Adaptation Highlights

1. **PyTorch for ROCm**: `setup_rocm.sh` auto-detects ROCm and installs `torch==2.5.1+rocm6.1`, verifying `torch.cuda.is_available()` returns True.
2. **Hunyuan3D-2 on ROCm**: Use `torch.float16` and `device='cuda'` (ROCm is exposed through PyTorch's CUDA compatibility layer). `Hunyuan3DDiTFlowMatchingPipeline.to()` is an in-place operation; we fixed misuse that caused a `None` pipeline.
3. **Texture module fallback**: Hunyuan3D-2's `custom_rasterizer` and `differentiable_renderer` depend on CUDA extensions that fail to compile on ROCm; the system automatically falls back to Trimesh-based front-projection texturing to keep the pipeline usable.
4. **Celery memory backend**: Demo environment has no Redis; enable `CELERY_TASK_ALWAYS_EAGER=true` and force broker/backend to `memory://` / `cache+memory://` to avoid Redis connection failures.
5. **VRAM optimization**: Enable `enable_model_cpu_offload` for Stable Diffusion and Zero123; 48 GB VRAM can simultaneously hold stylization, multi-view, and 3D generation models.

5.3 Performance Data

- 2.5D relief: photo to STL/GLB in about 2–5 minutes (depending on image resolution).
- Full-color 3D: photo to multi-view + 3D mesh + texture in about 10–30 minutes (first run requires model download).

- All key inference steps run locally on the AMD GPU without calling closed-source online APIs.

---

6. Innovation Highlights

- **Lazy Canvas workflow**: abstracts the complex 3D production pipeline into 6 visual nodes, lowering the learning curve while preserving Agent-modifiable flexibility.
- **3D Director Agent**: one-sentence generation of style, parameters, and flow plan, without manually selecting each model node.
- **Multimodal + printable closed loop**: image-to-image, multi-view, image-to-3D, 2.5D relief, print checking, Web UI preview, and download in one tool.
- **ROCm local-first**: runs Hunyuan3D-2 / Zero123 / Depth Anything V2 on AMD GPU, with a texture fallback solution.